🔹 1. Dependency Injection

🔍 Summary:

FastAPI has a powerful and simple-to-use **dependency injection system**. It lets you define reusable components (like DB sessions, auth validators, etc.) and "inject" them into routes or other dependencies using `Depends()`.

This leads to **cleaner, modular, and testable code**.

👨‍💼 Q&A:

Q1: What is dependency injection in FastAPI?
A1: It's a way to declare dependencies (like a database session, security checks, or reusable logic) that get automatically provided by FastAPI using the `Depends` function.

Q2: How do you create a dependency in FastAPI?
A2: Define a function and use `Depends(function)` in the route. For example:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/items/")
def read_items(db=Depends(get_db)):
    return db.query(Item).all()
```

Q3: Can dependencies be async?
A3: Yes, dependencies can be `async def` functions. FastAPI will handle them correctly.

Q4: How do you handle shared dependencies across multiple routes?
A4: Create a function with `Depends` and reuse it in multiple route handlers or dependencies.

🔹 2. Path Operations

🔍 Summary:

Path operations define the **API endpoints** in FastAPI. You use decorators like `@app.get()`, `@app.post()`, etc., to associate routes with Python functions.

They handle HTTP methods and route parameters.

👨‍💼 Q&A:

Q1: What are path operations in FastAPI?
A1: They are route definitions using decorators like `@app.get("/items/{id}")`. Each one maps an HTTP method and a URL path to a Python function.

Q2: Can you define multiple path parameters in a route?
A2: Yes. Example:

```
@app.get("/users/{user_id}/items/{item_id}")
def get_item(user_id: int, item_id: int):
    return {"user_id": user_id, "item_id": item_id}
```

Q3: How do you add metadata like tags or summaries to routes?
A3:

```python
@app.get("/items", tags=["Items"], summary="Get all items")
def get_items():
    ...
```

Q4: How does FastAPI validate path parameters?
A4: It uses Python type hints. For example, `item_id: int` ensures it must be an integer.

🔹 3. Request Body Models

🔍 Summary:

FastAPI uses **Pydantic models** to define and validate request bodies. These models describe the expected data format and types.

This leads to automatic validation and clear API docs.

👨‍💼 Q&A:

Q1: How do you accept request bodies in FastAPI?
A1: Define a Pydantic model and use it as a parameter.

```python
class Item(BaseModel):
    name: str
    price: float

@app.post("/items/")
def create_item(item: Item):
    return item
```

Q2: What happens if the input body doesn't match the model?
A2: FastAPI returns a 422 Unprocessable Entity with details about the validation error.

Q3: Can you use nested models?
A3: Yes. Pydantic supports nested models:

```python
class Owner(BaseModel):
    name: str

class Item(BaseModel):
    name: str
    owner: Owner
```

Q4: How do you make a field optional?
A4: Use `Optional` or provide a default value:

```python
description: Optional[str] = None
```

🔹 4. Query Parameters

🔍 Summary:

Query parameters are used to pass data in the URL like `/items?limit=10`. FastAPI extracts and validates them based on function parameters.

👨‍💼 Q&A:

Q1: How are query parameters handled in FastAPI?
A1: Any function parameter not part of the path becomes a query parameter by default.

```python
@app.get("/items/")
def get_items(limit: int = 10):
    return {"limit": limit}
```

Q2: How do you define optional query parameters?
A2: By setting a default value or using `Optional`:

```python
def get_items(skip: int = 0, limit: Optional[int] = None)
```

Q3: Can you add metadata like description to query parameters?
A3:

```python
from fastapi import Query

def get_items(q: str = Query(None, min_length=3, description="Search query")):
```

Q4: Can query parameters be lists?
A4: Yes. FastAPI can parse repeated query parameters as lists.

```python
def get_items(tags: List[str] = Query([])):
```

🔹 5. Background Tasks

🔍 Summary:

FastAPI supports **background tasks** using the `BackgroundTasks` class. These are lightweight functions that run **after sending the response** to the client.

It's perfect for **sending emails, logging, or processing** after the main request is complete.

👨‍💼 Q&A:

Q1: What is a background task in FastAPI?
A1: It's a function that runs after the response is returned. It's useful for non-blocking operations like sending emails or cleanup.

Q2: How do you declare a background task?
A2:

```python
from fastapi import BackgroundTasks

def write_log(message: str):
    with open("log.txt", "a") as f:
        f.write(message)

@app.post("/notify/")
def notify_user(background_tasks: BackgroundTasks):
    background_tasks.add_task(write_log, "User notified\n")
    return {"message": "Notification sent"}
```

Q3: Can background tasks be async?
A3: Yes, FastAPI will await them if they are async.

Q4: Are background tasks suitable for heavy processing?
A4: No, for long-running or CPU-intensive tasks, use something like **Celery** or a job queue.

🔹 6. Security

🔍 Summary:

FastAPI provides built-in tools for **security and authentication**, including **OAuth2**, **HTTP Basic Auth**, and **API key validation**, using dependencies.

It simplifies handling security in a modular and declarative way.

👨‍💼 Q&A:

Q1: How is security implemented in FastAPI?
A1: Using dependency injection and security utilities like `OAuth2PasswordBearer`, `HTTPBasic`, or custom token validation with `Depends`.

Q2: How do you secure a route using a token?
A2:

```
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.get("/protected/")
def protected_route(token: str = Depends(oauth2_scheme)):
    return {"token": token}
```

Q3: Can you define reusable auth dependencies?
A3: Yes. You can encapsulate token decoding/validation in a function and reuse it via `Depends()`.

Q4: Does FastAPI provide automatic security docs?
A4: Yes. When you use FastAPI's security utilities, the OpenAPI docs automatically show **Authorize** buttons.

🔹 7. FastAPI with DB

🔍 Summary:

FastAPI can easily integrate with SQL and NoSQL databases. The common stack uses **SQLAlchemy** for ORM and **Pydantic** for validation. Database sessions are often provided via dependency injection.

👨‍💼 Q&A:

Q1: What's the typical DB setup with FastAPI?
A1: Use SQLAlchemy + SessionLocal + Alembic for migration. Inject DB session with a dependency like `get_db()`.

Q2: Example of DB session dependency?
A2:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/users/")
def read_users(db: Session = Depends(get_db)):
    return db.query(User).all()
```

Q3: How do you handle schema validation in DB models?
A3: SQLAlchemy handles DB models; Pydantic models are used for request/response validation.

Q4: Can DB operations be async?
A4: Yes, with **databases** or **SQLModel** or **async SQLAlchemy**, though traditional SQLAlchemy is sync.

🔹 8. CORS (Cross-Origin Resource Sharing)

🔍 Summary:

CORS is a security feature that controls **which domains can access your API**. FastAPI uses `CORSMiddleware` to configure this behavior.

👨‍💼 Q&A:

Q1: Why do we need CORS in APIs?
A1: To control which frontend apps (from different domains) can make requests to your backend. Browsers enforce CORS as a security measure.

Q2: How to enable CORS in FastAPI?
A2:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # or specific domains
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Q3: Is `allow_origins=["*"]` safe?
A3: Not for production APIs. It opens your API to all domains. Prefer whitelisting your trusted frontend domains.

Q4: Does CORS affect server-to-server communication?
A4: No. CORS is a **browser-based security feature**, not enforced in server-to-server or Postman requests.

🔹 9. Middleware

🔍 Summary:

Middleware in FastAPI is a function that runs **before and after every request**. It's useful for **logging, error handling, request transformation, CORS (low-level), and more**.

FastAPI uses `@app.middleware("http")` for custom HTTP middleware.

👨‍💼 Q&A:

Q1: What is middleware in FastAPI?
A1: Middleware is code that executes before and after a request is processed. It can modify the request/response or perform tasks like logging, metrics, and header injection.

Q2: How do you define middleware in FastAPI?
A2:

```
@app.middleware("http")
async def log_requests(request: Request, call_next):
    print(f"Request: {request.url}")
    response = await call_next(request)
    print(f"Response status: {response.status_code}")
    return response
```

Q3: What is `call_next()`?
A3: It forwards the request to the actual path operation function and returns the response back up the stack.

Q4: Can middleware be used for authentication?
A4: It's not recommended for auth logic. Use dependencies for that. Middleware is more suited for global tasks.

🔹 10. WebSockets

🔍 Summary:

FastAPI supports **WebSocket connections** for **real-time communication** like chat apps, notifications, live dashboards, etc.

You define WebSocket endpoints using `@app.websocket` .

👨‍💼 Q&A:

Q1: What is a WebSocket in FastAPI?
A1: It's a two-way communication channel that remains open, allowing the server and client to send messages anytime.

Q2: How to define a WebSocket route?
A2:

```
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    await websocket.send_text("Hello WebSocket!")
    await websocket.close()
```

Q3: How to handle messages from the client?
A3:

```
data = await websocket.receive_text()
```

Q4: Can you have multiple clients connected?
A4: Yes, you can maintain a list of active connections and broadcast messages.

🔹 11. OAuth2 & JWT

🔍 Summary:

FastAPI has built-in support for **OAuth2 with password flow** and **JWT (JSON Web Tokens)** for securing APIs. OAuth2 handles user authentication, and JWT is used for securely transmitting the user identity.

👨‍💼 Q&A:

Q1: What is OAuth2 with password flow?
A1: It's a flow where users provide username and password directly to the client app, which then gets a token from the server. Suitable for first-party apps.

Q2: What is JWT and how is it used in FastAPI?
A2: JWT is a token format used to encode user data. FastAPI encodes user info into a JWT, which is passed in the `Authorization` header and decoded to authenticate users.

Q3: How to generate a JWT in FastAPI?
A3:

```
from jose import jwt

def create_access_token(data: dict, secret_key: str, expire_delta: timedelta):
    to_encode = data.copy()
    to_encode["exp"] = datetime.utcnow() + expire_delta
    return jwt.encode(to_encode, secret_key, algorithm="HS256")
```

Q4: How to protect routes using JWT?
A4: Use `Depends()` on a function that extracts and verifies the JWT token.

🔹 12. Pydantic

🔍 Summary:

FastAPI uses **Pydantic** for data validation. Pydantic models are used to define request bodies, responses, and internal schemas. It ensures data integrity using Python types.

👨‍💼 Q&A:

Q1: What is Pydantic used for in FastAPI?
A1: It's used for validating and parsing request/response data. It creates strict schemas using Python types.

Q2: How do you define a Pydantic model?
A2:

```
from pydantic import BaseModel

class User(BaseModel):
    username: str
    age: int
```

Q3: Can Pydantic models include validation rules?
A3: Yes, using field constraints or custom validators.

```
from pydantic import Field

class Product(BaseModel):
    name: str
    price: float = Field(gt=0)
```

Q4: How to convert SQLAlchemy objects to Pydantic models?
A4: Use `.from_orm()` with `Config.orm_mode = True`.

🔹 13. Response Models

🔍 Summary:

FastAPI lets you define **response models** using Pydantic to control and validate the **output** returned by your API. It ensures **clean, secure, and documented responses**.

👨‍💼 Q&A:

Q1: What is a response model in FastAPI?
A1: It's a Pydantic model that defines the structure of the response returned by an endpoint.

Q2: Why use a response model separately from request model?
A2: To **hide sensitive fields** like passwords or internal metadata, and to **control the API output** format.

Q3: How to declare a response model?
A3:

```
class UserOut(BaseModel):
    id: int
    username: str

@app.get("/users/{user_id}", response_model=UserOut)
def read_user(user_id: int):
    ...
```

Q4: How do you hide fields from response even if they exist in the object?
A4: Use a response model that doesn't include those fields. You can also use `.dict(exclude=...)`.

🔹 14. FastAPI Events

🔍 Summary:

FastAPI allows you to define **startup and shutdown events** using decorators. Useful for tasks like connecting to databases, loading ML models, or closing resources cleanly.

👨‍💼 Q&A:

Q1: What are FastAPI events?
A1: They're hooks to run logic **before the app starts** (`@app.on_event("startup")`) or **before it shuts down** (`@app.on_event("shutdown")`).

Q2: Common use cases of startup events?
A2: Initialize DB connections, load ML models, setup Redis cache.

Q3: Can startup/shutdown events be async?
A3: Yes, both sync and async functions are supported.

Q4: Example of event hook?
A4:

```
@app.on_event("startup")
async def startup_event():
    connect_to_db()
```

🔹 15. Rate Limiting

🔍 Summary:

FastAPI doesn't have built-in rate limiting, but it can be implemented using **custom middleware** or external packages like `slowapi` or by integrating **Redis**-based logic.

👨‍💼 Q&A:

Q1: Does FastAPI support rate limiting natively?
A1: No, but you can integrate it using packages like `slowapi` , `fastapi-limiter` , or custom middleware.

Q2: What is `slowapi` and how is it used?
A2: It's a FastAPI-compatible rate-limiter based on `limits` . Example:

```
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
@app.get("/items/")
@limiter.limit("5/minute")
def read_items():
    ...
```

Q3: How can Redis help in rate limiting?
A3: Redis can be used to store request counts with TTL (Time To Live) and track limits efficiently across multiple app instances.

Q4: Why is rate limiting important?
A4: It helps **prevent abuse, reduce load, and protect APIs** from overuse or DDoS-style attacks.

🔹 16. Form Validation

🔍 Summary:

FastAPI supports receiving form data using `Form(...)`. Useful for **HTML form submissions**, **login forms**, or **multipart content** (like forms with file uploads).

👨‍💼 Q&A:

Q1: How do you receive form data in FastAPI?
A1:

```
from fastapi import Form

@app.post("/login/")
def login(username: str = Form(...), password: str = Form(...)):
    return {"username": username}
```

Q2: What's the difference between Form and Body in FastAPI?
A2: `Form(...)` is for `application/x-www-form-urlencoded` content, while `Body(...)` is for `application/json`.

Q3: Can form data and files be sent together?
A3: Yes, using `Form` for fields and `File` for uploads.

Q4: Is Form input validated like JSON body?
A4: Yes, based on type hints. Invalid types will raise a 422 error.

🔹 17. OAuth2

🔍 Summary:

FastAPI supports **OAuth2 authorization flows**, particularly **Password Flow** for first-party apps. You use this to manage **token-based user authentication** and control access to secured endpoints.

👨‍💼 Q&A:

Q1: What is OAuth2 in FastAPI used for?
A1: OAuth2 is used for secure **user authentication** and **token generation**, allowing access to protected endpoints via bearer tokens.

Q2: What is the most commonly used OAuth2 flow in FastAPI?
A2: The **password grant flow**, typically used for internal apps (e.g., web + backend owned by same org).

Q3: How to define OAuth2 scheme in FastAPI?
A3:

```
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
```

Q4: Does FastAPI support OpenAPI security schema generation with OAuth2?
A4: Yes, it automatically adds `Authorize` buttons in Swagger docs when using OAuth2 schemes.

🔹 18. JWT Tokens

🔍 Summary:

JWT (JSON Web Tokens) are used to **encode user identity and roles** after login. They're stateless, self-contained, and widely used for authentication in modern APIs.

👨‍💼 Q&A:

Q1: What is a JWT and why is it used?
A1: JWT is a secure, base64-encoded token containing user claims (e.g., ID, role). It's used for stateless authentication.

Q2: What libraries are used for JWT in FastAPI?
A2: `python-jose` or `PyJWT` are commonly used.

Q3: How do you verify a JWT token in FastAPI?
A3:

```
from jose import jwt, JWTError

def decode_token(token: str):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        return payload
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")
```

Q4: How do you protect routes with JWT?
A4: Inject a dependency that decodes the token and validates the user, then pass that dependency in `Depends()`.

🔹 19. Mounting Static Files

🔍 Summary:

FastAPI allows mounting **static files** (HTML, CSS, JS, images) using `StaticFiles`. It's useful for serving frontends, uploaded files, or documentation pages.

👨‍💼 Q&A:

Q1: How to serve static files in FastAPI?
A1:

```
from fastapi.staticfiles import StaticFiles

app.mount("/static", StaticFiles(directory="static"), name="static")
```

Q2: What's the use of `name` in `app.mount()`?
A2: It helps FastAPI include it in the OpenAPI docs and internal route registry.

Q3: Can FastAPI serve frontend apps like React or Angular?
A3: Yes, just mount the `build` or `dist` folder as a static directory.

Q4: How to serve uploaded files?
A4: Save files in a folder (e.g., `uploads/`) and mount that path using `StaticFiles`.

🔹 20. Upload File API

🔍 Summary:

FastAPI makes file uploads easy using the `File` class. It supports both **single and multiple file uploads**, with automatic parsing of `multipart/form-data`.

👨‍💼 Q&A:

Q1: How to upload a file in FastAPI?
A1:

```
from fastapi import File, UploadFile
```

```python
@app.post("/upload/")
def upload(file: UploadFile = File(...)):
    return {"filename": file.filename}
```

Q2: What's the difference between `File` and `UploadFile`?
A2: `UploadFile` is better for large files since it uses Python's **SpooledTemporaryFile** for efficient streaming and memory use.

Q3: How to handle multiple file uploads?
A3:

```python
def upload(files: List[UploadFile] = File(...)):
    ...
```

Q4: How do you save an uploaded file?
A4:

```python
with open(f"files/{file.filename}", "wb") as f:
    f.write(file.file.read())
```

🧠 First, What's the Magic Behind All This?

🔧 Under the Hood: FastAPI uses Python type hints + function signature inspection + Starlette's request lifecycle to:

- **Parse** the incoming request
- **Match** your route handler
- **Inspect** the function parameters and **their type hints**
- Decide:
  - Is this a body param (Pydantic)?
  - Is this a dependency (via `Depends`)?
  - Is this a query/header/path param?

✅ 1. How Pydantic Validation Works (Even Without Depends)

🎯 Key Point:

If you add a **Pydantic model as a function parameter**, FastAPI automatically knows it's a **request body** and will **instantiate and validate it**.

📦 Example:

```python
class User(BaseModel):
    name: str
    age: int

@app.post("/users/")
def create_user(user: User):  # <- no Depends used
    return user
```

🕵️ What Happens Behind the Scenes:

1. FastAPI checks the parameter type: `user: User`.
2. It sees that `User` is a subclass of `BaseModel`.
3. Since it's **not a path/query/header/file**, FastAPI assumes it's from the **request body**.
4. It:
   - Reads the body

- Parses JSON
- Validates with Pydantic
- If valid → passes `User(...)` to the route
- If invalid → raises **422 Unprocessable Entity** with a structured error response

---

🤔 Why no `Depends` here?

Because FastAPI uses **parameter type inspection** to distinguish between:

- Dependencies (`Depends(...)`)
- Body models (`BaseModel`)
- Query/path/header/etc. (`str` , `int` , `Header` , `Query` , etc.)

✅ 2. How `Depends()` Injection Works

🔧 Mechanism:

If FastAPI sees `Depends(...)` , it knows:

- The value **should come from another function**
- That function may return a value (e.g., DB session, token, etc.)
- It **recursively resolves** any dependencies inside it

📦 Example:

```
def get_db():
    return "db-session"

@app.get("/items/")
def read_items(db=Depends(get_db)):
    return {"db": db}
```

🔍 Under the Hood:

1. FastAPI sees `db=Depends(get_db)`
2. It:
   - Calls `get_db()` before the route
   - Passes return value as `db`
   - Supports `yield` if it's a generator (for teardown)

If `get_db` itself had `Depends(...)` , FastAPI resolves **those too**. This is **recursive dependency injection**.

✅ 3. How FastAPI Handles Query, Header, Path, and Form Params

📦 Example:

```
from fastapi import Query, Header, Path, Form

@app.post("/form/")
def login(username: str = Form(...), password: str = Form(...)):
    ...
```

🔍 What's Happening:

- FastAPI inspects:

    - The function param **type** (`str` , `int` , etc.)
    - The **default value** being used (`Form(...)` , `Query(...)` , `Header(...)` , etc.)
- Based on this, FastAPI:
    - Knows where to pull the value from:
        - `Form(...)` → from `multipart/form-data`
        - `Query(...)` → from URL query string
        - `Header(...)` → from HTTP headers
        - `Path(...)` → from path parameters

These are **special classes** under `fastapi.params` , which act as **markers** telling FastAPI **where to extract the value from**.

🎯 So What Tells FastAPI What To Do?

Clue in Signature	Source	Extraction Method
`user: User` (BaseModel)	Request Body	Use `.json()` , validate with Pydantic
`db=Depends(...)`	Dependency Function	Call it, resolve recursively
`id: int = Path(...)`	URL Path	Extract from URL pattern
`q: str = Query(...)`	Query String	Extract from `?q=value`
`auth: str = Header(...)`	HTTP Header	Extract from headers
`file: UploadFile = File(...)`	File Upload	Extract from multipart/form-data
`username: str = Form(...)`	HTML Forms	Extract from form data

🧠 TL;DR — Why It Just Works

- FastAPI uses Python's **function signature inspection** (via `inspect` module)
- It knows how to **parse each parameter** based on:
    - **Type hints** (e.g., `User`)
    - **Default value classes** (`Depends` , `Query` , `Header` , etc.)
- Uses **Pydantic** under the hood for validation
- Returns:
    - `422` → Validation failed
    - `401` / `403` → Custom exception (via `HTTPException`)
    - Custom responses based on your logic
